Reviewer Pack — Minimal Rank of Free Probability Distributions vs BP_ReadTwi…

Entry ea528936a40b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 21:19:43 UTC

Minimal Rank of Free Probability Distributions vs BP_ReadTwice Circuit Size

Entry ID: `ea528936a40b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 21:19:43 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Complexity Theory: BP_ReadTwice Circuit Complexity

Statement:

> For every read-twice branching program P with size n, the minimal rank of its free probability distribution is upper-bounded by O(n^(1/3)). Specifically, there exists a constant c > 0 such that for all n ≤ 40, the inequality Rank(ρ(P)) ≤ cn^(1/3) holds, where ρ(P) is the free probability distribution of P.

Rationale (proposer's reasoning):

> Free probability theory provides a noncommutative generalization of classical probability theory, which has applications in quantum information and operator algebras. If the minimal rank of a read-twice branching program's free probability distribution can be shown to be polynomially bounded, it may reveal new structural properties of these programs that are not accessible through traditional complexity measures.

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `acd4661b1d1b5335`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given n ≤ 40, if the maximum rank among 30 randomly generated read-twice branching programs' free probability distributions is less than or equal to 1.5 times the median rank, then the conjecture is supported. Otherwise, it is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - free probability AND minimal rank AND BP_ReadTwice circuit complexity - O(n^(1/3)) bound ON free probability distributions AND read-twice circuit size - Rank(ρ(P)) ≤ cn^(1/3) inequality IN free probability theory AND BP_ReadTwice circuit

Top relevant hits considered: - [http://arxiv.org/abs/1001.2131v1] Asymptotics of the probability minimizing a "down-side" risk - [http://arxiv.org/abs/2204.11679v2] Eigenstate Thermalization Hypothesis and Free Probability - [http://arxiv.org/abs/1506.00166v2] Optimal Investment to Minimize the Probability of Drawdown - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/2007.11292v2] First observation of the decay $\Lambda_b^0 \to \eta_c(1S)pK^-$ - [http://arxiv.org/abs/2407.14261v2] Study of charmonium production via the decay to $p\bar{p}$ at $\sqrt{s} = 13TeV$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
```

```
        return A

    def rank(A):
        A = gaussian_elimination(A)
        r = 0
        for row in A:
            if any(row):
                r += 1
        return r

    def read_twice_branching_program(n):
        program = []
        for _ in range(n):
            state = random.randint(0, 1)
            transition = random.choice(['left', 'right'])
            program.append((state, transition))
        return program

    def free_probability_distribution(program):
        n = len(program)
        A = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n):
            state, transition = program[i]
            if state == 0:
                A[0][i+1] += 1
            else:
                A[n][i+1] += 1
            if transition == 'left':
                A[i+1][i+2] += 1
            else:
                A[i+1][i] += 1
        return rank(A)

    n = random.randint(5, 40)
    program = read_twice_branching_program(n)
    rank_value = free_probability_distribution(program)

    return {
        "metric_name": "Rank",
        "metric_value": rank_value,
        "instances_tested": 1,
        "conjecture_holds": rank_value <= 1.5 * (n ** (1/3)),
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_rank = sum(result["metric_value"] for result in results)
    mean_rank = total_rank / len(results)
```

```
    std_rank = math.sqrt(sum((result["metric_value"] - mean_rank) ** 2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    else:
        first_failing_seed = next(result["seed"] for result in results if not result["conjecture_h
        counterexample = "First failing seed"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce18b478.py", line 87, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce18b478.py", line 72, in run_trial
    rank_value = free_probability_distribution(program)
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce18b478.py", line 65, in free_probabil
    A[i+1][i+2] += 1
    ~~~~~~^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11357
2	propose	ollama_remote	glm4:latest	0	0	10074

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	preregistration	ollama_remote	glm4:latest	0	0	5444
4	novelty	ollama_remote	glm4:latest	0	0	5069
5	novelty	ollama_remote	glm4:latest	0	0	6124
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25720
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18872
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11427
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11672
10	judge	ollama_remote	glm4:latest	0	0	42263

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 148020 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/ea528936a40b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ea528936a40b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ea528936a40b.tar.gz` (if generated)